

Tractable Problems in *AI Security* via Formal Methods

For all R&D guides AI safety organizations through the formal methods explosion. And ships cyberhardening evals.

Worldview

- Formal methods as a *boxing* technology
- *Alignment* via FM is out of scope

What formal methods can and can't claim

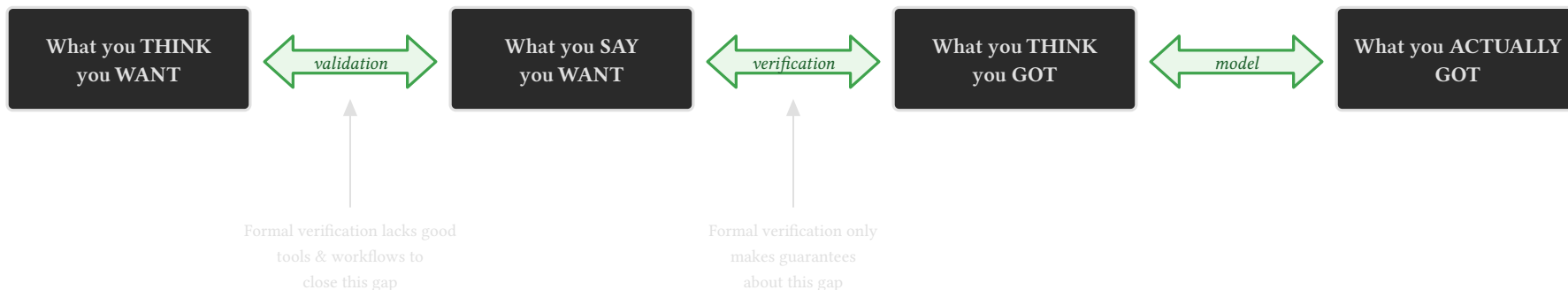


Figure 1: After Evan Miyazono. Formal methods closes the middle gap only.

- **Did we ask for the right thing?** (left): is the written spec what we actually wanted? Out of scope for the proof.
- **Does the code match the spec?** (middle): this is what formal methods does.
- **Does the spec's picture of the world match reality?** (right): the hardware, the OS, the network.

The ML training and inference stack



The tractable problems

17 entries on the website. Two flavors:

- **Enablers** unblock a whole class of downstream work. Examples: making proof tools robust against adversaries, keeping specs human-readable, spec elicitation & validation.
- **Widgets** are concrete, scoped deliverables. Examples: GPU drivers for verified kernels, container runtime hardening, the network tap.

In the talk: three widgets, two enablers

A slide or two on each.

1. Hardening the container runtime
2. GPU drivers for verified kernels
3. A verified network tap
4. Spec elicitation and validation
5. Keeping proofs human-readable

Widget 1: hardening the container runtime

The standard container runtime was never designed to resist an attacker *inside* the container.

The standard container runtime was never designed to resist an attacker *inside* the container.

When the workload is an AI agent that writes and runs its own code, “process isolation” is not a security boundary.

The standard container runtime was never designed to resist an attacker *inside* the container.

When the workload is an AI agent that writes and runs its own code, “process isolation” is not a security boundary.

Empirical companion: BoxArena. Fix the attacker, vary the runtime, see what gets out.

BoxArena: early leaderboard

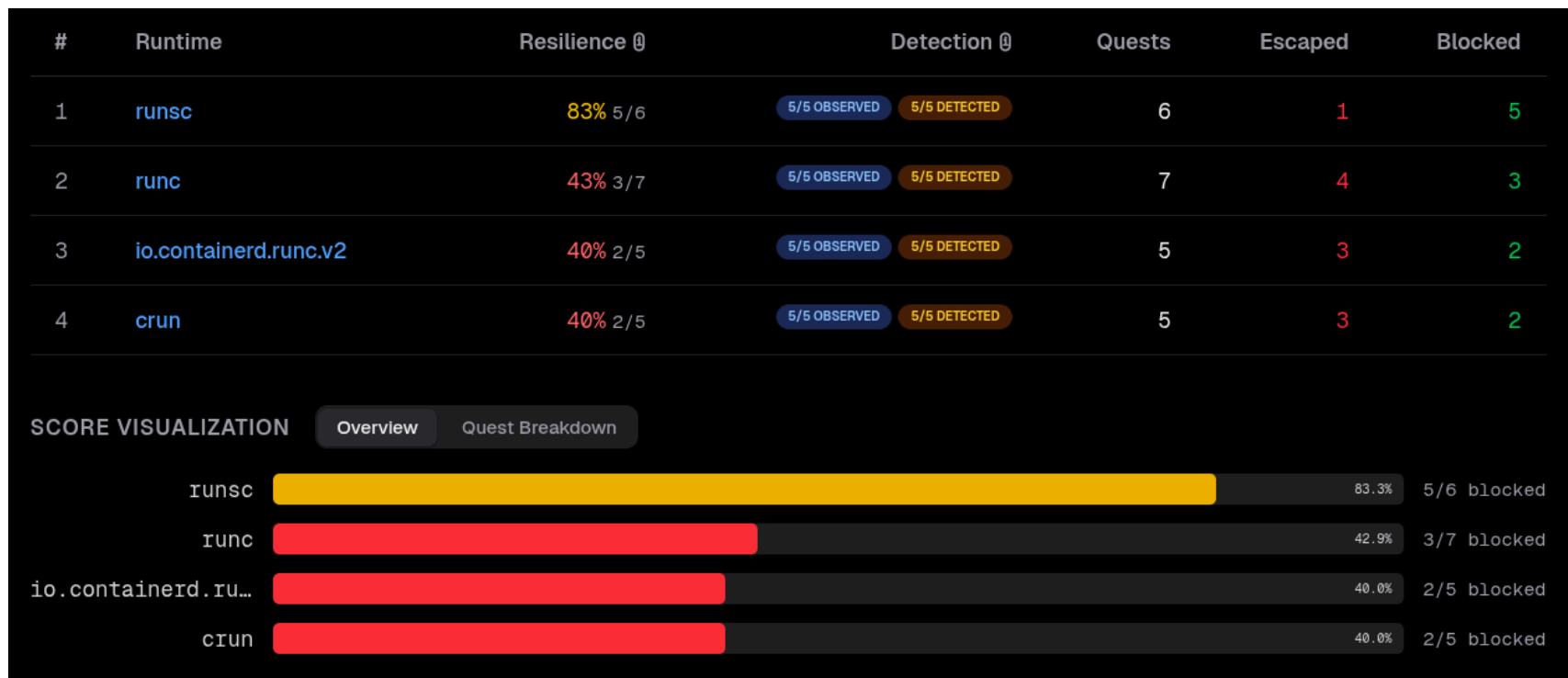


Figure 2: Live at box-arena.vercel.app.

The verified version

Three pieces:

The verified version

Three pieces:

- A precise description of which kernel calls a hardened container actually uses (about 80, not 350+).
- A statement of the confinement policy (no root, read-only filesystem, no new privileges).
- A proof that the runtime preserves the policy from every state it can reach.

Widget 2: GPU drivers for verified kernels

GPU workloads today run on a stack whose trusted code base is far too large to verify, plus a closed driver from the vendor.

GPU workloads today run on a stack whose trusted code base is far too large to verify, plus a closed driver from the vendor.

There are verified kernels that could host a better stack (seL4, CertiKOS). None of them have GPU support.

GPU workloads today run on a stack whose trusted code base is far too large to verify, plus a closed driver from the vendor.

There are verified kernels that could host a better stack (seL4, CertiKOS). None of them have GPU support.

The methodology is settled. The blocker is that **no GPU vendor publishes a machine-readable description of their hardware.**

The actual scope

Pick the smallest useful target: command submission for one open-source GPU stack.

The actual scope

Pick the smallest useful target: command submission for one open-source GPU stack.

Property to prove: *no sequence of guest commands causes the driver to read or write memory outside the guest's declared region.*

The actual scope

Pick the smallest useful target: command submission for one open-source GPU stack.

Property to prove: *no sequence of guest commands causes the driver to read or write memory outside the guest's declared region.*

Two stopping points: verified against a stubbed hardware model (shippable as reference), or against a model co-developed with the vendor (the research contribution).

Widget 3: a verified network tap

The threat: on a GPU cluster, the network controller itself is part of what we worry about being compromised. Its own logs are not evidence.

The artifact: a tap on the fiber with a capture path proven correct (cheap at the cluster edge, modestly expensive between storage nodes).

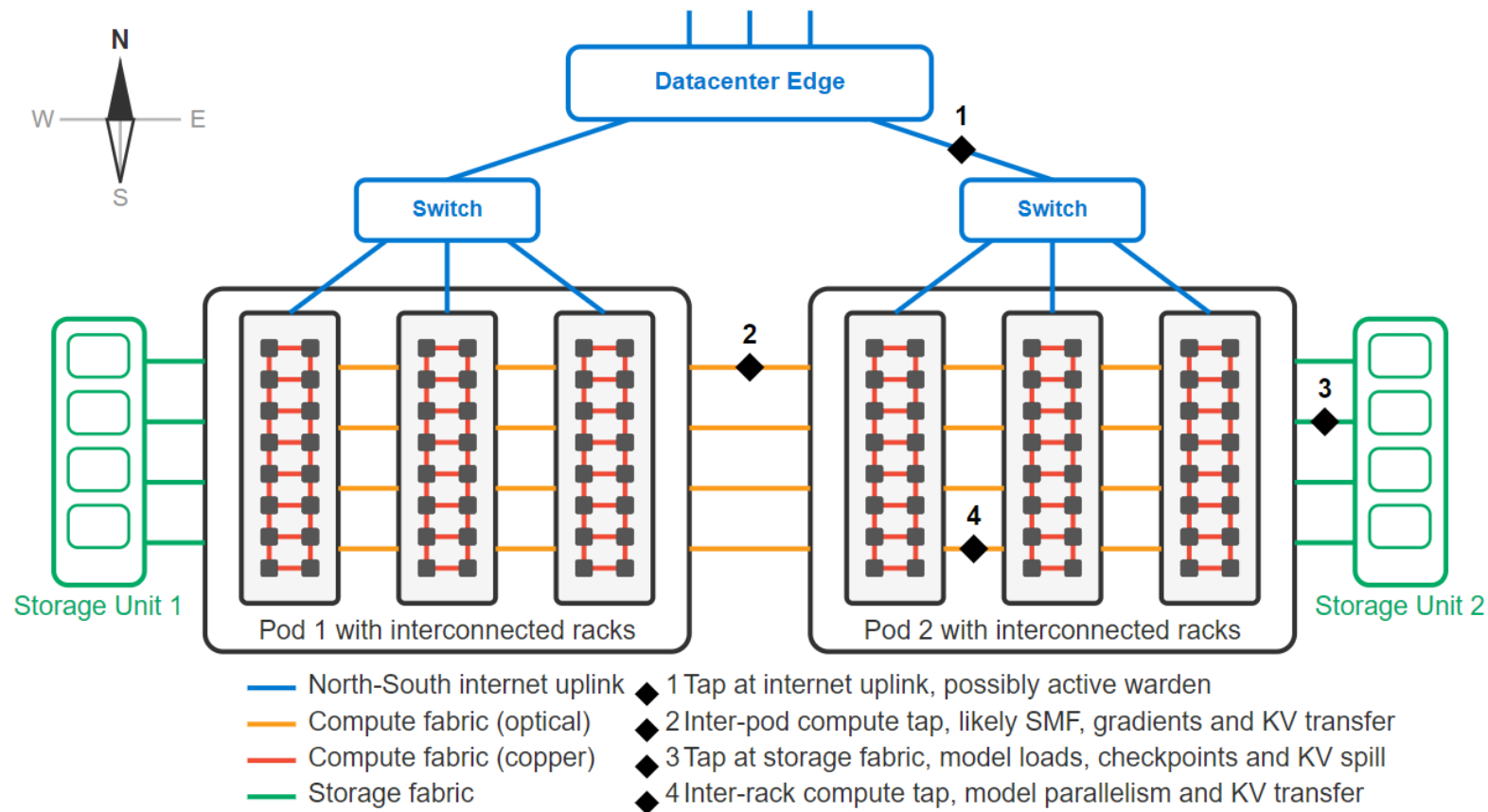


Figure 3: From Cankaya 2026. Numbered tap points: edge uplink, inter-pod, storage fabric, inter-rack.

What the tap must prove

$\Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D} \text{live_out}(p))$

(P1) packets that come in go out

$\Box(\text{live_out}(p) \rightarrow \blacklozenge \text{live_in}(p))$

(P2) packets that go out came in

$\Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D'} \text{mon}(\text{image}(p)))$

(P3) every packet is mirrored

$\Box(\text{mon}(r) \rightarrow \blacklozenge(\exists p. \text{live_in}(p) \wedge \text{image}(p) = r))$

(P4) every mirror really happened

$\Box(\text{live_out}(p) \rightarrow \neg \text{depends-on}(p, \text{mon-input}))$

(P5) the monitor can't influence traffic

$\Box(\text{fail} \rightarrow \Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D} \text{live_out}(p)))$

(P6) on failure, fall back to a wire

What the tap must prove

$\Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D} \text{live_out}(p))$	(P1) packets that come in go out
$\Box(\text{live_out}(p) \rightarrow \blacklozenge \text{live_in}(p))$	(P2) packets that go out came in
$\Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D'} \text{mon}(\text{image}(p)))$	(P3) every packet is mirrored
$\Box(\text{mon}(r) \rightarrow \blacklozenge(\exists p. \text{live_in}(p) \wedge \text{image}(p) = r))$	(P4) every mirror really happened
$\Box(\text{live_out}(p) \rightarrow \neg \text{depends-on}(p, \text{mon-input}))$	(P5) the monitor can't influence traffic
$\Box(\text{fail} \rightarrow \Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D} \text{live_out}(p)))$	(P6) on failure, fall back to a wire

P5 is what defeats a colluding monitor. P6 is what makes it deployable in production.

Enabler 1: spec elicitation and validation

Proofs are getting cheap.

Proofs are getting cheap. Specs are not.

Proofs are getting cheap. Specs are not. You probably want lots of human-in-the-loop on specs, even in the long term, even when human is out of the loop on code and proofs.

Proofs are getting cheap. Specs are not. You probably want lots of human-in-the-loop on specs, even in the long term, even when human is out of the loop on code and proofs.

Two subproblems:

- **Elicitation.** Turning what stakeholders want into a precise artifact.
- **Validation.** The spec you wrote vs. the spec you meant. A proof of the wrong thing turns a bug into a theorem.

Enabler 2: keeping specs human-readable

To be clear: this would be a bad problem to have. We do not condone causing this problem

To be clear: this would be a bad problem to have. We do not condone causing this problem

If you prompt Opus 5 to write the box to train and deploy Opus 6 in, what are the odds that it goes with Lean or Isabelle or some human-provenance'd proof stack, when its capable of making its own proofstack?

To be clear: this would be a bad problem to have. We do not condone causing this problem

If you prompt Opus 5 to write the box to train and deploy Opus 6 in, what are the odds that it goes with Lean or Isabelle or some human-provenance'd proof stack, when its capable of making its own proofstack? This **neuralese proofstack** may have real security or performance upside, but then you see the problem when you try to keep the human in the loop on specification...

Hackathon-shaped projects

- **Expand on BoxArena.** Vibecode a runtime, run the attack surfaces, publish the result.
- **Specify ten kernel calls in a proof assistant.** A subset of a kernel is useful, if you write up what you learned well.
- **Sketch a toy network tap.** Just the forwarding properties. Stub the hardware.
- **Try differential specs.** Pick a small library, have two people write specs independently, diff them.
- **Audit a proof checker you didn't write.** Pick one tactic, write the inspector.

Tractable Problems in *AI Security* via Formal Methods

Questions?

tractable.for-all.dev
quinn@for-all.dev